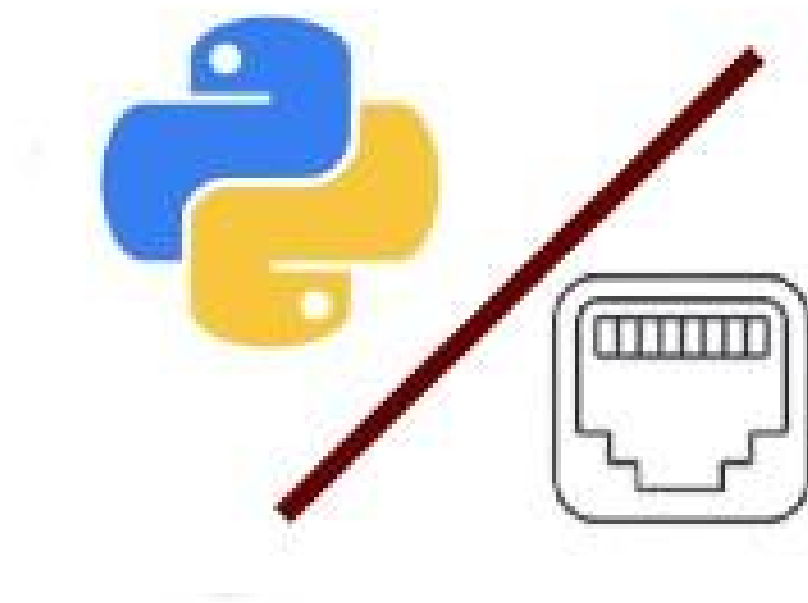


Projet Script Python: Scan de port & Analyseur de logs

Niveau du projet: Intermédiaire

Thème: Linux, python, sécurité informatique et automatisation

Type de document: Rapport



Date de début de rédaction: 18/08/2026

Date de fin de rédaction: 23/08/2026

Auteur: AKAKPO Like Axel

Sommaire:

1. Introduction
2. Développement
 - a. Intérêt du projet
 - b. limit du projet
 - c. reflex à avoir
3. Conclusion

1. Introduction:

Ce projet est un simple script qui se présente comme un outil ayant fonction la première scanner les ports d'une machine(que ce soit la nôtre ou une distante). Pour ce projet j'ai eu à parcourir diverses documentation forum et tutoriel qui seront mentionnés dans la partie annexe. Pour parler du développement de ce document après cette partie nous aborderons l'intérêt du projet, le pourquoi ce script peut vous être utile, les limites de ce script et aussi les réflexes à avoir après son usage puis nous concluons en quelque bref mots.

2. Développement:

a. Intérêt du projet:

Bon tout d'abord avant de chercher à comprendre l'intérêt, remettons au claire cette notion de base. Qu'est ce qu'un port tout d'abord? Car oui nous ne pouvons pas commencer sans savoir ce que l'on scan, donc un port c'est tout simplement un entier non signé de 16 bits qui permet d'indiquer avec précision là où les paquets doivent arrivée(en gros derrière un port on a une application), un port suit aussi généralement un autre élément qui est une adresse ip qui est simplement une suite de numéro ou lettre(dans le cas d'ipV6) séparé par un séparateur(“.” ou “:”) qui permet d'identifier de façon unique un appareil sur un réseau. Une dernière notion à connaître est celle de protocole réseau un protocole réseau est une règle qui définit la façon dont les ordinateurs communiquent entre eux; si on se réfère au modèle réseau(OSI ou TCP/IP) on a la couche de transport 2 principaux protocoles TCP et UDP, pour ne pas s'éterniser dans nos explications je dirais simplement que TCP vérifie que les deux machines peuvent communiquer avant d'autoriser les échanges de données et que les données sont arrivées(donc c'est lent) pendant que UDP lui ne vérifie rien de tout cela(donc c'est plus rapide) mais bien sûr ils ont d'autres fonctions mais principalement retenez que c'est cela qu'ils font. Maintenant que tout est expliqué nous devons savoir qu'est ce qu'un scanner de port, déjà dans le nom vous avez port donc comme expliqué plus haut un port permet d'envoyer des données à une application avec précision, ce qu'il faut savoir maintenant c'est que ce projet utilise le protocole TCP dans ce cas c'est à dire que l'on vérifie des connexions entre les machines(plus précisément entre les ports des machines) avant d'envoyer des données et c'est ce principe de vérification de connexion qui nous sera utile, quand en python on se connecte par tcp à un port la fonction utilisée nous renvoie un résultat qui est 0 si la connexion réussit une erreur si elle échoue donc nous allons demander à python de se connecter à ce qu'on appelle la plage des ports “well-known” un par un puis de se déconnecter à chaque fois donc le processus: notre script prend le port 1 - il se connecte - si la connexion réussit il affiche que le port est ouvert s'il échoue le port est fermé - il ferme la connexion, puis il fait pareil avec le port 2 puis 3 jusqu'au port 1023. En vérité avoir des ports ouverts n'est pas initialement un danger car c'est littéralement vitale pour la communication avec ton ordinateur, mais le danger c'est d'avoir des services non utilisés et donc qui n'ont probablement pas été configurés ou régulièrement mis à jour qui ont leur port ouvert, alors à chaque usage de ce script il faut toujours et toujours fermer les services inutiles, mais aussi petite précision importante avant que j'explique son utilisation ce script est à usage éthique si vous l'utilisez sur des machines qui ne vous appartiennent pas et dont vous n'avez l'autorisation peut vous causer selon la loi togolaise une Peine

d'emprisonnement : De 6 mois à 2 ans et une Amende : De 5 000 000 à 20 000 000 de francs CFA.

donc ne faite pas de chose illégaux avec, parenthèse étant fermé passons au explication du script:

```
python3 main.py -c [lp_cible] -p [port_cible] -pi
```

Voici la syntax possible avec ce script on peut très bien écrire un python3 [main.py](#) et cela fonctionnera sur des paramètres par défaut; l'option -c(ou —cible) sert à donné pour cible une ip autre que cela local, l'option -p(--port) sert à donné une précision sur le port que l'on cherche à scanner(plus tôt que scanne tout les port well know) et pi (ou —ping) est une fonction annexe pour vérifier la connectivité par ping.

b. Limit du projet:

Depuis tout à l'heure je vante les mérites de mon script mais il faut bien souligner quelque détail le premier est le suivant: ce script ne remplacera pas nmap! Car oui ici on scan tout à fait des ports c'est bien mais un outils aussi complet et avancé que nmap est un choix plus pertinent dans le sujet de la reconnaissance et pentest notamment pour la variété d'option et d'outils qu'il propose, par contre ce que mon projet offre est un code compréhensible et modifiable à bon vouloir selon les situation idéalement utilisé le en tant qu'admin réseau pour vérifier les ports de votre machine rapidement mais je déconseil vraiment à un pentester ou aspirant pentester de l'utiliser (principalement à cause des traces que ce script laisse dans les logs). La deuxième limite claire est que ce script ne touche pas au port udp il se contente uniquement de scanner les ports des services utilisant TCP.

c. Reflex à avoir

Bon pour être assez bref, le principal réflexe à avoir est de se demander des services que j'ai est ce tout d'abord est ce que c'est nécessaire que la communication soit accessible à distance ? Ensuite est ce que j'ai configuré de façon sécurisé ce service? Et pour finir est ce que la version de ce service n'est pas en vérité un risque? Si l'une de ces questions est non il vaudrait mieux fermer ce port ou au moins mettre un pare-feu par dessus.

3. Conclusion

En conclusion, ce projet m'a permis de mieux comprendre concrètement plusieurs notions fondamentales liées aux réseaux, notamment le fonctionnement des ports, des connexions TCP, des sockets et de la communication entre deux machines. La réalisation d'un scanner de ports, même relativement simple, permet de passer de la théorie à la pratique en observant directement comment un programme peut tenter d'établir une connexion avec différents services afin de déterminer leur état.

Ce projet possède cependant des limites importantes et ne prétend pas remplacer des outils professionnels comme Nmap. Son principal intérêt réside davantage dans son aspect pédagogique et dans sa capacité à être facilement compris, modifié et adapté à différents besoins. Il rappelle également qu'un port ouvert n'est pas nécessairement une vulnérabilité,

mais qu'un service inutile, mal configuré ou obsolète accessible sur le réseau peut constituer une surface d'attaque supplémentaire.

Enfin, l'utilisation d'un scanner de ports doit toujours s'accompagner d'une démarche de sécurité responsable : identifier les services réellement nécessaires, limiter leur exposition, les configurer correctement et maintenir leurs versions à jour. Un outil de sécurité ne constitue donc qu'un moyen d'observation ; la véritable sécurité dépend surtout de l'analyse des résultats et des mesures prises à la suite de cette analyse.

À travers ce projet, j'ai ainsi pu consolider mes compétences en Python, en programmation réseau et en sécurité informatique, tout en développant une meilleure compréhension du fonctionnement d'un scanner de ports. Cette expérience constitue une base qui pourra être approfondie par la suite avec l'ajout de nouvelles fonctionnalités, notamment la prise en charge d'autres protocoles, l'amélioration de la détection des services et le développement d'un véritable module d'analyse de logs.